



Forest Crawler - Documentation

MABROUK Sofia, SAYAVONG Melvin

ENSEA, 2025 - 2026, Semestre 6

Table des matières

1	Présentation	2
2	Class Main	2
3	Class Game Engine	2
3.1	Variables	2
3.2	keyTyped	2
3.3	keyPressed	3
3.4	keyReleased	3
4	RenderEngine	3
4.1	Variables	4
4.1.1	Screen settings	4
4.1.2	World settings	4
4.1.3	FPS	4
4.2	RenderEngine	4
4.3	startGameThread	4
5	Class Player	4
5.1	Variables (description)	5
5.2	Player	5
5.3	setDefaultValues	5
5.4	getPlayerImage	5
5.5	update	6
5.6	draw	7
6	Class Sprite	8
7	Class CollisionCheck	8
7.1	CollisionCheck	8
7.2	checkTile	9
8	Tile	10
9	TileManager	11
9.1	TileManager	11
9.2	tileSetup	11
9.3	getTileImage	11
9.4	loadMap	12
9.5	drawLayer	12
9.6	draw	14

1 Présentation

2 Class Main

La classe Main est la classe permettant d'exécuter tout le jeu, il s'occupe principalement de la gestion de la fenêtre.

```
1   JFrame window = new JFrame();
2   window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
3   window.setResizable(false);
4   window.setTitle("Forest Crawler");
```

Cette partie du code correspond aux paramètres de la fenêtre :

- `window.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`: permet de déterminer le comportement du bouton de fermeture de la fenêtre.
- `window.setResizable(false)`: permet de savoir si la taille de la fenêtre est modifiable ou non.
- `window.setTitle("Forest Crawler")`: correspond au nom de la fenêtre.

3 Class Game Engine

Cette classe s'occupe de la gestion du comportement des touches à l'aide d'un listener. Elle sert uniquement à configurer les touches pour le déplacement du joueur.

3.1 Variables

Cette classe permet de gérer le comportement des touches du clavier à l'aide d'un listener. Dans notre cas, on utilise les touches du clavier uniquement pour se déplacer donc on crée 4 variables booléennes :

- `upPressed`
- `downPressed`
- `leftPressed`
- `rightPressed`

3.2 keyTyped

```
1   @Override
2   public void keyTyped(KeyEvent keyEvent) {
3
4   }
```

Cette méthode permet de savoir si une touche a été appuyé puis relâché. Elle ne sert pas pour ce projet, mais est automatiquement créée par IntelliJ dans la classe KeyListener (qui est implémentée par GameEngine) avec `keyPressed` et `keyReleased`.

3.3 keyPressed

```
1      @Override
2      public void keyPressed(KeyEvent keyEvent) {
3          int code = keyEvent.getKeyCode();
4
5          if(code == KeyEvent.VK_Z){
6              upPressed = true;
7          }
8          if(code == KeyEvent.VK_Q){
9              leftPressed = true;
10         }
11         if(code == KeyEvent.VK_S){
12             downPressed = true;
13         }
14         if(code == KeyEvent.VK_D){
15             rightPressed = true;
16         }
17     }
```

Cette méthode permet de savoir si les touches liées au déplacement ont été appuyé.

3.4 keyReleased

```
1      @Override
2      public void keyReleased(KeyEvent keyEvent) {
3          int code = keyEvent.getKeyCode();
4          if(code == KeyEvent.VK_Z){
5              upPressed = false;
6          }
7          if(code == KeyEvent.VK_Q){
8              leftPressed = false;
9          }
10         if(code == KeyEvent.VK_S){
11             downPressed = false;
12         }
13         if(code == KeyEvent.VK_D){
14             rightPressed = false;
15         }
16     }
```

Cette méthode permet de savoir si les touches liées au déplacement ont été relâché.

4 RenderEngine

Cette classe correspond à la configuration du rendu du jeu au sein de la fenêtre comme la taille des éléments du jeu. Ce rendu dépend de deux groupes de paramètres de position : **screenXXX**, une position relative à la fenêtre de jeu, et **worldXXX**, une position par relative à la map du jeu, qui est 'out of bounds' de l'écran.

4.1 Variables

4.1.1 Screen settings

Voici la liste des différentes variables pour cette section :

- `originalTileSize` : Taille réelle des cases
- `scale` : Échelle d'agrandissement
- `tileSize` : Taille des cases lors de l'affichage
- `maxScreenCol` : Nombre de colonnes affichées sur la fenêtre de jeu
- `maxScreenRow` : Nombre de lignes affichées sur la fenêtre de jeu
- `screenWidth` : Longueur de la fenêtre de jeu
- `screenHeight` : Hauteur de la fenêtre de jeu

4.1.2 World settings

Voici la liste des différentes variables pour cette section :

- `maxWorldCol` : Nombre de colonnes de la map de jeu
- `maxWorldRow` : Nombre de lignes de la map de jeu
- `worldWidth` : Longueur maximum de la carte affichée
- `worldHeight` : Hauteur maximum de la carte affichée

4.1.3 FPS

La variable FPS correspond au nombre d'images par secondes.

4.2 RenderEngine

```
1 public RenderEngine(){
2     this.setPreferredSize(new Dimension(screenWidth , screenHeight));
3     this.setBackground(Color.decode("#55af32"));
4     this.setDoubleBuffered(true);
5     this.addKeyListener(keyH);
6     this.setFocusable(true );
7 }
```

Cette méthode permet de définir les différents paramètres de la fenêtre.

4.3 startGameThread

```
1 public void startGameThread(){
2     gameThread = new Thread(this);
3     gameThread.start();
4 }
```

Cette méthode permet d'initialiser le thread pour le jeu.

5 Class Player

Cette classe permet de gérer tous les attributs et comportements liés au joueur comme sa position, son déplacement ou son sprite.

5.1 Variables (description)

- `RenderEngine re` : instance de `RenderEngine`
- `GameEngine keyH` : instance de `GameEngine`
- `screenX` : position X du joueur sur la carte.
- `screenY` : position Y du joueur sur la carte.

5.2 Player

```
1      public Player(RenderEngine re, GameEngine keyH) {
2          this.re = re;
3          this.keyH = keyH;
4
5          screenX = re.screenWidth/2 - re.tileSize/2;
6          screenY = re.screenHeight/2 - re.tileSize/2;
7
8          //solidArea = new Rectangle(1/(6 * re.tileSize), 1/(3* re.tileSize),
9              2/(3 * re.tileSize), 2/(3 * re.tileSize));
10         solidArea = new Rectangle(16, 30, 32, 32);
11
12         setDefaultValues();
13         getPlayerImage();
14     }
```

Cette méthode correspond au constructeur de l'objet `Player`

- `solidArea` : `Rectangle` placé sur le joueur, qui sert de zone de collision (cf. `CollisionCheck`)
- `screenX`, `screenY` : Positionnement du joueur au centre de la fenêtre de jeu

5.3 setDefaultValues

```
1      public void setDefaultValues(){
2          worldX= re.tileSize * 27;
3          worldY = re.tileSize * 25 ;
4          speed=4;
5          direction="down";
6      }
```

Cette méthode permet de définir les valeurs par défaut lorsqu'on lance le jeu.

5.4 getPlayerImage

```
1      public void getPlayerImage(){
2          try{
3              up1 = ImageIO.read(getClass().getResourceAsStream("/characters/
4                  gnome/walk/gnome_z_walk1.png"));
5              up2 = ImageIO.read(getClass().getResourceAsStream("/characters/
6                  gnome/walk/gnome_z_walk2.png"));
7              down1 = ImageIO.read(getClass().getResourceAsStream("/characters/
8                  gnome/walk/gnome_s_walk1.png"));
9          }
```

```
6         down2= ImageIO.read(getClass().getResourceAsStream("/characters/
           gnome/walk/gnome_s_walk2.png"));
7         right1 = ImageIO.read(getClass().getResourceAsStream("/characters/
           gnome/walk/gnome_d_walk1.png"));
8         right2 = ImageIO.read(getClass().getResourceAsStream("/characters/
           gnome/walk/gnome_d_walk2.png"));
9         left1 = ImageIO.read(getClass().getResourceAsStream("/characters/
           gnome/walk/gnome_q_walk1.png"));
10        left2 = ImageIO.read(getClass().getResourceAsStream("/characters/
           gnome/walk/gnome_q_walk2.png"));
11
12        }catch(IOException e){
13            e.printStackTrace();
14        }
15    }
```

Cette méthode permet de chercher les sprites du joueur en fonction de sa direction. Il y a 2 sprites par direction pour donner l'illusion du mouvement lors du déplacement.

5.5 update

Cette méthode permet de vérifier à tout instant les évènements de déplacement et de collision ainsi que la mise à jour du sprite.

```
1    public void update(){
2        if(keyH.upPressed || keyH.downPressed || keyH.leftPressed || keyH.
           rightPressed) {
3
4            if (keyH.upPressed) {
5                direction = "up";
6            }
7            if (keyH.downPressed) {
8                direction = "down";
9            }
10           if (keyH.leftPressed) {
11               direction = "left";
12           }
13           if (keyH.rightPressed) {
14               direction = "right";
15           }
16
17           /* Check tile collision*/
18           collisionOn = false;
19           re.cCheck.checkTile(this);
20           if (collisionOn == false) {
21               switch (direction) {
22                   case "up":
23                       worldY -= speed;
24                       break;
25                   case "down":
26                       worldY += speed;
27                       break;
28                   case "left":
29                       worldX -= speed;
30                       break;
31                   case "right":
```

```
32         worldX += speed;
33         break;
34     }
35 }
36
37 spriteCounter++;
38 if (spriteCounter > 10) {
39     if (spriteNumber == 1) {
40         spriteNumber = 2;
41     } else if (spriteNumber == 2) {
42         spriteNumber = 1;
43     }
44     spriteCounter = 0;
45 }
46
47 }
48 }
```

5.6 draw

Cette méthode permet de mettre à jour la bonne image en fonction de la direction, en animant le mouvement.

```
1  public void draw(Graphics2D g2 ){
2      //g2.setColor(Color.white);
3      //g2.fillRect(x, y , re.tileSize, re.tileSize);
4      BufferedImage image = null;
5
6      switch(direction) {
7          case "up":
8              if (spriteNumber == 1){
9                  image = up1;
10             }
11             if (spriteNumber == 2){
12                 image = up2;
13             }
14             break;
15          case "down":
16              if (spriteNumber == 1){
17                  image = down1;
18             }
19             if (spriteNumber == 2){
20                 image = down2;
21             }
22             break;
23          case "right":
24              if (spriteNumber == 1){
25                  image = right1;
26             }
27             if (spriteNumber == 2){
28                 image = right2;
29             }
30             break;
31          case "left":
32              if (spriteNumber == 1){
33                  image = left1;
```



```
34         }
35         if (spriteNumber == 2){
36             image = left2 ;
37         }
38         break;
39     }
40
41     g2.drawImage(image, screenX, screenY, re.tileSize, re.tileSize, null);
42
43     // DEBUG: Draw the Hitbox
44     g2.setColor(Color.RED);
45     g2.drawRect(screenX + solidArea.x, screenY + solidArea.y, solidArea.
46         width, solidArea.height);
47 }
```

6 Class Sprite

Cette classe permet de configurer le comportement des sprites.

```
1     public class Sprite {
2         public int worldX, worldY ;
3         public int speed;
4         public BufferedImage up1, up2, down1, down2, right1, right2, left1,
5             left2, idle1, idle2;
6         public String direction;
7
8         public int spriteCounter = 0;
9         public int spriteNumber = 1;
10
11         public Rectangle solidArea;
12         public boolean collisionOn = false;
13     }
```

Voici la description des différentes variables :

- **worldX, worldY** : Position X et Y du sprite sur la carte
- **speed** : Vitesse du sprite sur la carte
- **up, down, right, left, idle** : Variables permettant d'utiliser les bonnes images en fonction de la direction
- **direction** : Direction du sprite parmi [up, down, left, right]
- **spriteCounter** : Compteur permettant d'actualiser les images pour l'animation
- **spriteNumber** : Permet de basculer entre les différentes images lors d'une animation
- **solidArea** : Hitbox du sprite
- **collisionOn** : Activation de la collision

7 Class CollisionCheck

Cette classe permet de faire la gestion des collisions avec les différents sprite.

7.1 CollisionCheck

Constructeur pour définir le RenderEngine.

7.2 checkTile

```
1 public void checkTile(Sprite sprite){
2     int spriteLeftWorldX = sprite.worldX + sprite.solidArea.x;
3     int spriteRightWorldX = sprite.worldX + sprite.solidArea.x + sprite.
        solidArea.width;
4     int spriteTopWorldY = sprite.worldY + sprite.solidArea.y;
5     int spriteBottomWorldY = sprite.worldY + sprite.solidArea.y + sprite.
        solidArea.height;
6
7     int spriteLeftCol = spriteLeftWorldX/re.tileSize;
8     int spriteRightCol = spriteRightWorldX/re.tileSize;
9     int spriteTopRow = spriteTopWorldY/re.tileSize;
10    int spriteBottomRow = spriteBottomWorldY/re.tileSize;
11
12    int tileBottomNum1, tileBottomNum2, tileTopNum1, tileTopNum2;
13
14    switch(sprite.direction){
15        case "up":
16            spriteTopRow = (spriteTopWorldY - sprite.speed) / re.tileSize;
17            tileBottomNum1 = re.tileM.mapBottomNum[spriteLeftCol][
                spriteTopRow];
18            tileBottomNum2 = re.tileM.mapBottomNum[spriteRightCol][
                spriteTopRow];
19            tileTopNum1 = re.tileM.mapTopNum[spriteLeftCol][spriteTopRow
                ];
20            tileTopNum2 = re.tileM.mapTopNum[spriteRightCol][spriteTopRow
                ];
21
22            if(re.tileM.tile[tileBottomNum1].collision == true || re.tileM
                .tile[tileBottomNum2].collision == true ||
23                re.tileM.tile[tileTopNum1].collision == true || re.
                tileM.tile[tileTopNum2].collision == true) {
24                sprite.collisionOn = true;
25            }
26            break;
27        case "down":
28            spriteBottomRow = (spriteBottomWorldY + sprite.speed) / re.
                tileSize;
29            tileBottomNum1 = re.tileM.mapBottomNum[spriteLeftCol][
                spriteBottomRow];
30            tileBottomNum2 = re.tileM.mapBottomNum[spriteRightCol][
                spriteBottomRow];
31            tileTopNum1 = re.tileM.mapTopNum[spriteLeftCol][
                spriteBottomRow];
32            tileTopNum2 = re.tileM.mapTopNum[spriteRightCol][
                spriteBottomRow];
33
34            if(re.tileM.tile[tileBottomNum1].collision == true || re.tileM
                .tile[tileBottomNum2].collision == true ||
35                re.tileM.tile[tileTopNum1].collision == true || re.
                tileM.tile[tileTopNum2].collision == true) {
36                sprite.collisionOn = true;
37            }
38            break;
39        case "left":
```

```
40         spriteLeftCol = (spriteLeftWorldX - sprite.speed) / re.  
            tileSize;  
41         tileBottomNum1 = re.tileM.mapBottomNum[spriteLeftCol][  
            spriteTopRow];  
42         tileBottomNum2 = re.tileM.mapBottomNum[spriteLeftCol][  
            spriteBottomRow];  
43         tileTopNum1 = re.tileM.mapTopNum[spriteLeftCol][spriteTopRow];  
44         tileTopNum2 = re.tileM.mapTopNum[spriteLeftCol][  
            spriteBottomRow];  
45  
46         if(re.tileM.tile[tileBottomNum1].collision || re.tileM.tile[  
            tileBottomNum2].collision ||  
47             re.tileM.tile[tileTopNum1].collision || re.tileM.tile[  
                tileTopNum2].collision) {  
48             sprite.collisionOn = true;  
49         }  
50         break;  
51     case "right":  
52         spriteRightCol = (spriteRightWorldX + sprite.speed) / re.  
            tileSize;  
53         tileBottomNum1 = re.tileM.mapBottomNum[spriteRightCol][  
            spriteTopRow];  
54         tileBottomNum2 = re.tileM.mapBottomNum[spriteRightCol][  
            spriteBottomRow];  
55         tileTopNum1 = re.tileM.mapTopNum[spriteRightCol][spriteTopRow  
            ];  
56         tileTopNum2 = re.tileM.mapTopNum[spriteRightCol][  
            spriteBottomRow];  
57  
58         if(re.tileM.tile[tileBottomNum1].collision || re.tileM.tile[  
            tileBottomNum2].collision ||  
59             re.tileM.tile[tileTopNum1].collision || re.tileM.tile[  
                tileTopNum2].collision) {  
60             sprite.collisionOn = true;  
61         }  
62         break;  
63     }  
64 }
```

Cette méthode permet de déterminer si la hitbox touche une sprite de collision en fonction de la direction du joueur. Illustrons par un exemple : si le joueur va vers le haut (`case "up"`), alors les deux coins supérieurs de la hitbox sera comparée au bas du sprite se situant en haut du joueur. Ensuite, si un de ces coins et le bas du sprite se touchent, il y a collision : `sprite.collisionOn = true`

8 Tile

Cette classe permet de gérer l’affichage des cases ainsi que le comportement de celle-ci vis-à-vis du joueur. La class Tile contient 4 variables :

- `image` : prends l’image correspondante
- `collision` : Permet de définir si l’image devra être une collision ou non
- `isOverhead` : Permet de définir si l’image devra être au dessus du joueur ou non, (cf. méthode `draw` de `TileManager`)
- `width`, `height` : Définit la taille de la case

9 TileManager

Cette classe permet de gérer les sprites et de dessiner la map. Elle dessine en réalité deux maps se superposant : la première avec des sprites de background, la deuxième avec des sprites 'assets' de type objets / décorations, qui ne font pas 32x32 pixels.

9.1 TileManager

Cette méthode est le constructeur de la classe TileManager

```
1      public TileManager(RenderEngine re){
2          this.re = re;
3          tile = new Tile[100];
4          mapTopNum = new int[re.maxWorldCol][re.maxWorldRow];
5          mapBottomNum = new int[re.maxWorldCol][re.maxWorldRow];
6          getTileImage();
7          loadMap("/maps/village/village_bottom.txt", mapBottomNum);
8          loadMap("/maps/village/village_top.txt", mapTopNum);
9      }
```

9.2 tileSetup

Cette méthode permet d'importer une sprite en une ligne plutôt que six, ce qui rendrait le code trop long. La sprite est paramétrée comme suit : son index, son path dans le projet, si c'est une sprite de collision et si c'est une sprite à dessiner au dessus du joueur.

```
1      public void tileSetup(int index, String filePath, boolean collision,
2                          boolean isOverhead) {
3          System.out.println("Attempting to load: " + filePath);
4          try {
5              //Debug
6              //      if (getClass().getResourceAsStream(filePath) == null) {
7              //          System.out.println("File not found: " + filePath);
8              //      }
9              tile[index] = new Tile();
10             tile[index].image = ImageIO.read(getClass().getResourceAsStream(
11                 filePath));
12             tile[index].collision = collision;
13             tile[index].isOverhead = isOverhead;
14             tile[index].width = tile[index].image.getWidth();
15             tile[index].height = tile[index].image.getHeight();
16         } catch (IOException e) {
17             e.printStackTrace();
18         }
19     }
```

9.3 getTileImage

Cette classe agit comme une bibliothèque des sprites du jeu, qui sont instaurées avec la méthode `tileSetup`.

```
1      public void getTileImage(){
2          /*Sprites du chemin*/
3          // LIGNES DROITES
4          tileSetup(0, "/background/Forest/chemin/Horiz.png", false, false);
5          tileSetup(1, "/background/Forest/chemin/Vertical.png", false, false);
6          // COINS / VIRAGES SIMPLES
7
8          ...
```

9.4 loadMap

Cette classe permet de charger la map à partir d'un fichier .txt qui contient 50 lignes de 50 nombres, chaque nombre correspondant à l'index d'une sprite.

```
1      public void loadMap(String mapPath, int mapNum[][]){
2
3          try{
4              InputStream is = getClass().getResourceAsStream(mapPath);
5              BufferedReader br = new BufferedReader(new InputStreamReader(is));
6
7              int col = 0;
8              int row = 0;
9
10             while(col < re.maxWorldCol && row < re.maxWorldRow){
11                 String line = br.readLine();
12                 while(col < re.maxWorldCol){
13                     String numbers[] = line.split(" ");
14                     int num = Integer.parseInt(numbers[col]);
15                     mapNum[col][row] = num;
16                     col++;
17                 }
18                 if(col == re.maxWorldCol){
19                     col = 0;
20                     row ++;
21                 }
22             }
23             br.close();
24         } catch (Exception e) {
25         }
26
27     }
```

9.5 drawLayer

Cette méthode permet de dessiner la map, et ce en fonction des mouvements du joueur, qui reste au centre de la fenêtre de jeu pendant que les sprites se redessinent, pour créer la caméra du jeu.

```
1      public void drawLayer(Graphics2D g2, int mapNum[][], boolean not32,
2          boolean isOverhead) {
3          int worldCol = 0;
4          int worldRow = 0;
```

```
5      try {
6          while (worldCol < re.maxWorldCol && worldRow < re.maxWorldRow) {
7              int tileNum = mapNum[worldCol][worldRow];
8
9              if(tile[tileNum] == null){
10                  System.out.println("Missing tile: " + tileNum +
11                      " at col=" + worldCol + " row=" + worldRow);}
12
13              if(tileNum != 99) {
14
15                  if (tile[tileNum].isOverhead == isOverhead) {
16
17                      boolean isLarge = (tile[tileNum].image.getWidth(null)
18                          > 32 ||
19                          tile[tileNum].image.getHeight(null) > 32);
20
21                      if (isLarge == not32) {
22                          int Width = tile[tileNum].image.getWidth(null);
23                          int Height = tile[tileNum].image.getHeight(null);
24                          int scaledWidth = Width * re.scale;
25                          int scaledHeight = Height * re.scale;
26                          int worldX = worldCol * re.tileSize;
27                          int worldY = worldRow * re.tileSize;
28
29                          /*Camera*/
30                          int screenX = worldX - re.player.worldX + re.
31                              player.screenX;
32                          int screenY = worldY - re.player.worldY + re.
33                              player.screenY;
34
35                          /*Drawing only sprites in the screen*/
36                          if (worldX > re.player.worldX - re.player.screenX
37                              - re.tileSize && worldX < re.player.worldX +
38                                  re.player.screenX + re.tileSize
39                              && worldY > re.player.worldY - re.player.
40                                  screenY - re.tileSize && worldY < re.
41                                  player.worldY + re.player.screenY + re.
42                                  .tileSize) {
43                              g2.drawImage(tile[tileNum].image, screenX,
44                                  screenY, scaledWidth, scaledHeight, null);
45                          }
46                      }
47                  }
48              }
49              worldCol++;
50
51              if (worldCol == re.maxWorldCol) {
52                  worldCol = 0;
53                  worldRow++;
54              }
55          }
56      } catch (Exception e) {
57          throw new RuntimeException(e);
58      }
59  }
```

9.6 draw

Cette méthode permet de dessiner en superposant les deux maps, avec d'abord la map `bottom` puis la map `top`.

```
1 public void draw(Graphics2D g2, boolean isOverhead){
2     /*Code offered by the video tutorial, however it didn't work for
3       sprites that weren't 32x32, it shrunk them or they were overdrawn
4       by other sprites,
5       which is why I wrote drawLayer method*/
6     /*int x = 0;
7       int y = 0;
8
9     while(col < re.maxScreenCol && row < re.maxScreenRow){
10        int tileNum = mapTileNum[col][row];
11
12        g2.drawImage(tile[tileNum].image, x, y, re.tileSize, re.tileSize,
13            null);
14        col++;
15        x += re.tileSize;
16
17        if(col == re.maxScreenCol){
18            col = 0;
19            x = 0;
20            row++;
21            y += re.tileSize;
22        }
23    }
24
25    drawLayer(g2, mapBottomNum, false, isOverhead); /* Draws 32x32
26    background sprites first */
27    drawLayer(g2, mapBottomNum, true, isOverhead); /* Draws sprites of
28    other dimensions */
29    drawLayer(g2, mapTopNum, false, isOverhead); /* Draws 32x32 background
30    sprites first */
31    drawLayer(g2, mapTopNum, true, isOverhead); /* Draws sprites of other
32    dimensions */
33 }
```